Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

Concepts from class that may be helpful in programming my website are if statements, while loops, and time.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think that the click counter will be the easiest part of the project because that is a pretty simple program to build. I think that managing docstrings and code clarity will be the hardest part of the project because I am very bad at naming variables.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I think that creating the automatic clickers will require the most time and effort. The clicking counter and store for upgrades should be implemented first.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Distractions that could interfere with my work are my phone, instagram, and people. When I am working on this project I will try to work in a silent place with minimal distractions.

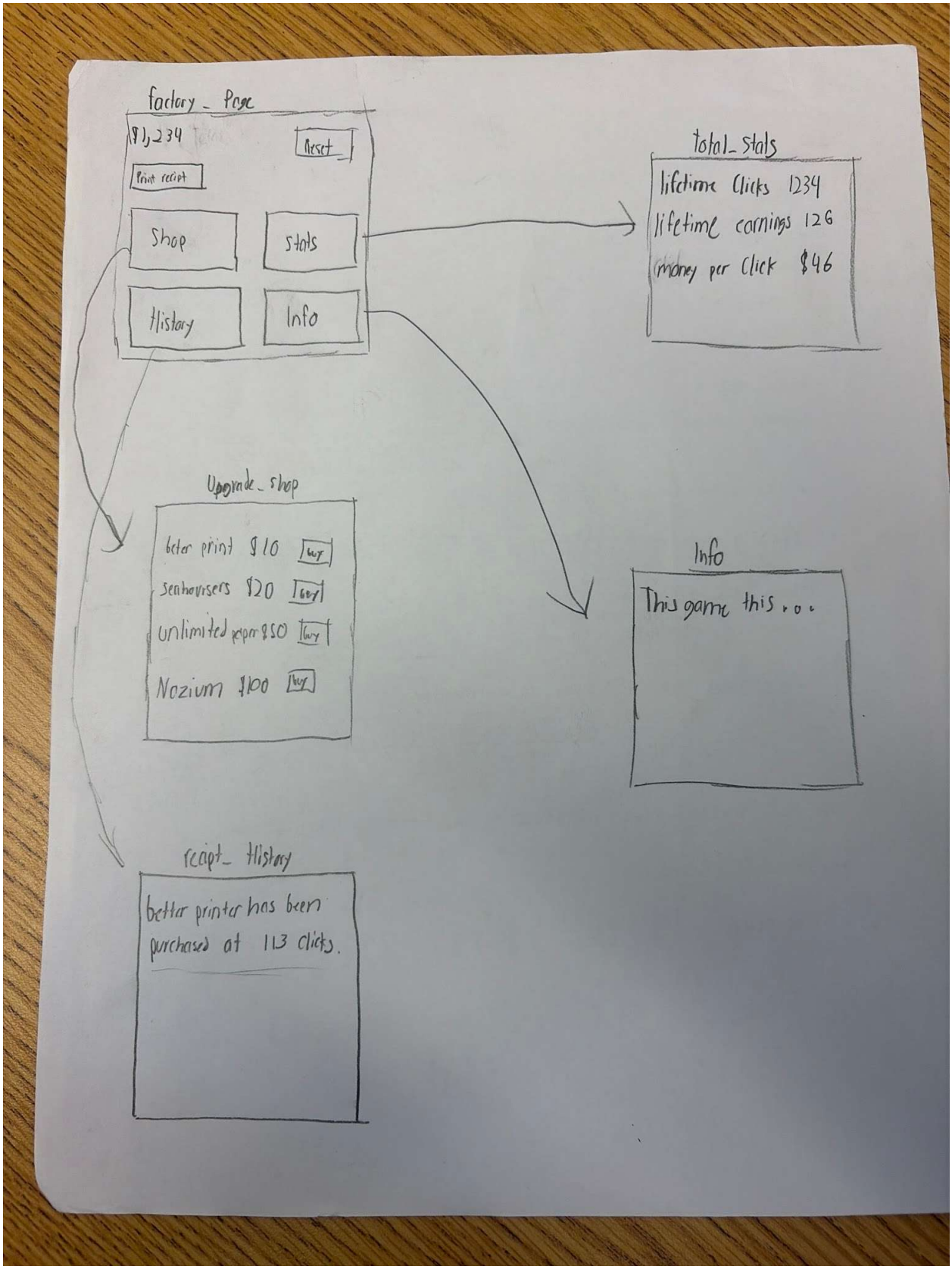
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across

the entire website.



State dataclass
money - int :
Total_earned - int:
Total_clicks - int:
Money_per_click - int
Current_upgrades - list[Upgrade]:
Company_name - str: t

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**The purpose of this website is to create a fun clicker game that is creative and entertaining.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	money - int : Represents the current amount of money that the user is able to spend Total_earned - int: Tracks the cumulative money that has been earned when playing the game Total_clicks - int: Counts the total number of clicks that have been made Money_per_click - int ;Tracks the amount of money that is earned for each click Current_upgrades - list[Upgrade]: Lists Representing the current upgrades the player has bought. Company_name - str: the current name of the company doing the clicking. Named by the player
Second Dataclass Your second dataclass, including	Upgrade Name - str: Contains the name of the upgrade

Requirement	**Implementation**
its attributes, their types, and how they will be meaningfully modified (if they are modified)	Cost_purchase- int: How much the upgrade will cost to purchase Money_per_click - int: total money per click after the upgrade has been bought Bought- bool: if the upgrade has been bought by the player
Routes (minimum 7) The names of your route functions	factory_page: Displays the click button, money, and overall game. add_click: Adds money and clicks to the main counter Shop_page: Has the upgrades available for purchase and their prices purchase_upgrade: Checks if the player can afford the upgrade and if so adds it to the game stats_page : Houses lifetime earnings, receipts printed, and upgrades purchased. receipt_history: Displays players current upgrades game_reset : Resets the count and upgrades to the beginning state
Pages (minimum 5) A brief description of the pages in your site	Factory_page: The main page where the player clicks, earns money and gets to the other pages Upgrade_shop: The page where the player is able to purchase upgrades for the clicking button.

Requirement	Implementation
	Total_stats: The page that houses the stats of total money earned, and lifetime clicks. Receipt_History The page that has the purchased upgrades by the player Info: The page contains info on how to play the game
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	TextBox: Lets the player enter their own company name. SelectBox: Allows the player to select a company department such as business, big business, or bigger business CheckBox: Lets the player to enable or disable milestone text
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	Statement 1: Checking if players can afford upgrades by comparing the money the player has to the money needed for the upgrade. Statement 2: Checking if the player has enough money to win the game. (Win condition) Will display a win message when tripped. Statement 3: For displaying text statements once the player has reached big milestones like $1000 or $10,000
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in	This loop will iterate over the list of upgrades and display and change them as the player purchases them. This is meaningful because it allows us to

Requirement	Implementation
your State attribute. Description of this loop and why it is meaningful to the site	show each upgrade without having to write separate code for each one.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Pasted text(2).txt

Document

Give me some ideas on how to fill out the outline for this clicker game

Requirement Implementation

Purpose/Functionality

A 1-2 sentence description of the purpose or functionality of your site

State Dataclass Attributes

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.

Second Dataclass

Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)

Routes (minimum 7)

The names of your route functions

Pages (minimum 5)

A brief description of the pages in your site

User Input (minimum 3)

Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes

For more information on these input types, see .

If Statements (minimum 3)

Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site

Loop (minimum 1)

This loop must iterate over the list/dict of dataclass instances in your State attribute.

Description of this loop and why it is meaningful to the site

…

I think The Receipt That Never Ends is a really good clicker game idea for this assignment. I'd keep the absurd corporate theme, but make the actual programming fairly simple.

Based on your outline, you need:

At least 4 State attributes.

A second dataclass.

At least 7 routes.

At least 5 pages.

At least 3 user input components.

At least 3 meaningful if statements.

At least 1 loop that iterates through your list of dataclass instances.

The biggest thing I'd recommend is making the game itself simple while using the other pages for things like a shop, statistics, and game information.

Here is how I'd fill out your outline.

Suggested outline: The Receipt That Never Ends

1. Purpose/Functionality

The purpose of the website is to create an absurd incremental clicker game where players run a company that makes money by printing increasingly unnecessary receipts. Players earn money by clicking a button, purchasing upgrades, and expanding their receipt-printing business until they can print a receipt for the entire universe.

2. State Dataclass Attributes

I'd suggest these 6 attributes:

Attribute

Type

Purpose

money

int

Tracks how much money the player currently has. Increases when printing receipts and decreases when buying upgrades.

total_earned

int

Tracks the total money earned throughout the game, even when money is spent.

receipts_printed

int

Tracks how many receipts the player has printed.

money_per_click

int

Determines how much money the player earns every time they print a receipt.

upgrades

list[Upgrade]

Stores all the available upgrades and their information, including whether they have been purchased.

company_name

str

Stores the current company name, which can change as the player progresses.

You only need four, but I'd use six because they give you more opportunities to demonstrate meaningful state changes.

3. Second Dataclass

I'd call the second dataclass Upgrade.

Each upgrade represents something the player can purchase to improve their receipt-printing business.

Attribute

Type

Purpose

name

str

The name of the upgrade, such as "Suspiciously Long Paper."

cost

int

How much money the upgrade costs.

bonus

int

How much the upgrade increases money earned per click.

description

str

A funny description explaining what the upgrade does.

purchased

bool

Tracks whether the player has purchased the upgrade.

Some example upgrades:

Slightly Better Printer

Starter upgrade
Citizen CBM-910II (CBM91024PFA2)
Cost: $10

Bonus: +$1 per click

Description: "It jams slightly less often. Management calls this innovation."

Suspiciously Long Paper

Cost: $50

Bonus: +$5 per click

Description: "The receipt is now longer than the actual shopping trip."

Unpaid Intern

Cost: $100

Bonus: +$10 per click

Description: "Has been told that printing receipts builds character."

Accounting Department

Cost: $500

Bonus: +$25 per click

Description: "Now charges the company for printing its own receipts."

Important: I'd initially make all upgrades increase money per click. This is much easier than implementing automatic income, timers, and background processes. You can add passive income later if you have time.

4. Routes (minimum 7)

You don't need seven completely different pages. Some routes can perform actions and return the player to the same page.

Here are seven routes I'd use:

Route function

Purpose

home_page

Displays the main game, money, and print button.

print_receipt

Adds money and increments the receipt counter.

shop_page

Displays available upgrades and their prices.

buy_upgrade

Checks whether the player can afford an upgrade and purchases it.

stats_page

Displays lifetime earnings, receipts printed, and upgrades purchased.

receipt_history_page

Displays information about the player's purchased upgrades.

reset_game

Resets the player's progress to the beginning.

You could also have an about_page if you want another route.

5. Pages (minimum 5)
I would organize the website into five pages:

ReceiptHero: Digital Receipts
1. Main Game Page

The main page where players print receipts, earn money, and see their current balance. It also displays a funny company message that changes as they progress.

Shop – The Regional Assembly of Text | The Shopkeepers
2. Upgrade Shop

Displays all available upgrades, their costs, descriptions, and purchase buttons. Players can spend their money to increase their earnings.

아직도 엑셀로 사업을 운영하시나요? 진짜 비용이 얼마인지 확인하세요 | FastDX

3. Company Statistics

Shows the player's total money earned, receipts printed, upgrades purchased, and current company name.

Long Shopping List: A Waste of Paper

4. Receipt History

Shows the upgrades the player has purchased, along with funny descriptions of their business accomplishments.

An Employee handbook near notepads and pen.

5. About / Instructions

Explains how to play, the purpose of the company, and the ultimate goal of printing the receipt for everything.

You can have navigation buttons at the top of each page so players can move between them.

6. User Input (minimum 3)

This part is a little tricky because clicker games don't naturally need much user input. I'd incorporate a few simple inputs that fit the game.

Input component

Purpose

TextBox

Allows the player to enter a custom company name.

SelectBox

Allows the player to select a company department or business title, such as

Accounting, Printing, or Management.

CheckBox

Allows the player to enable or disable funny company announcements.

These inputs don't need to change the core game mechanics. They just add personalization.

For example:

TextBox: Player names their company "Definitely Not a Scam Inc."

SelectBox: Player selects their department.

CheckBox: Player chooses whether to display random corporate messages.

7. If Statements (minimum 3)

Here are some meaningful conditionals that would naturally fit the game.

If statement 1: Checking whether the player can afford an upgrade.

If the player's money is greater than or equal to the upgrade cost, allow the purchase.

Otherwise, display a message saying they don't have enough money.

If statement 2: Unlocking new company stages.

If lifetime earnings reach $1,000, change the company name to "Receipt Conglomerate."

If lifetime earnings reach $100,000, display a new company announcement.

If lifetime earnings reach $1,000,000, unlock the final objective.

If statement 3: Checking whether the player has won.

- If lifetime earnings reach $10,000,000 and they have purchased the Universal Paper Roll, display the winning message.

Optional fourth: Checking whether the player has enabled company announcements.

If the announcement checkbox is enabled, display the funny company memo.

Otherwise, hide it.

8. Loop (minimum 1)
This is where your upgrades list becomes important.

You will use a for loop to iterate through every upgrade in the player's State dataclass.

For each upgrade, the program will:

Display its name.

Display its cost.

Display its description.

Check whether it has been purchased.

Display whether it is available for purchase or already owned.

This loop is meaningful because it allows you to display every upgrade without having to write separate code for each one.

For example, conceptually:

```
for upgrade in state.upgrades:
    # Display the upgrade's name
    # Display its cost
    # Display its description
    # Display whether it is purchased
```

This is probably the most natural way to satisfy the loop requirement.

How I'd organize the actual game
Here's the structure I'd aim for:

TOTALLY NORMAL RECEIPTS INC.
OFFICIAL COMPANY CHECKOUT SYSTEM
CURRENT COMPANY BALANCE
$1,250
Lifetime receipts: 342

PRINT RECEIPT
Click to generate more unnecessary paperwork.
LATEST COMPANY MEMO

"Management has approved printing receipts for the receipts used to print receipts."

Shop

Purchase upgrades
Statistics

View progress
History

View purchases
Instructions

Learn the rules

Illustrative layout, not a working game.

My recommended scope for a CS1 project

I'd divide your features into three categories so you don't accidentally make the project too complicated.

Feature

Priority

Clicking to earn money

Essential

Buying upgrades

Essential

Displaying money and receipts printed

Essential

Upgrade dataclass and list

Essential

Five pages and seven routes

Essential

Company milestones and messages

Essential

Company name customization

Essential

Receipt history

If time allows

Automatic printing with timers

Stretch goal

Saving progress with local storage

Stretch goal

Random events

Skip for now

One important Drafter consideration: Your game state should remain in the State dataclass as players navigate between pages. Make sure your route functions receive and return the appropriate state. You shouldn't need a database or backend for this game.

A few decisions I'd make before submitting your outline

There are three things I'd like you to think about because they will affect how easy the project is to build.

1. How should upgrades work?

Increase money earned per click (simplest)

Generate money automatically over time (more complicated)

Both manual clicking and automatic income

2. How do you want to handle the five pages?

Five separate pages with navigation

Fewer pages, with some information combined

One main game page and several supporting pages

3. How should upgrades be purchased?

Each upgrade can only be purchased once

Upgrades can be purchased multiple times, with increasing costs

My personal recommendation is to use money per click, five simple pages, and upgrades that can only be purchased once for your first version. That gives you enough complexity to demonstrate your programming skills without making the project overwhelming.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.